



中国科学技术大学
University of Science and Technology of China

网络空间安全学院
School of Cyber Science and Technology

作品类别: ☒ 软件设计 ☐ 硬件制作 ☐ 工程实践

《密码学导论》课程大作业作品设计报告

作品题目: 基于 NIST STS 的伪随机数统计分布测评与蒙特卡罗应用

团队名称: rank

团队人员: 明宜霖 (个人)

2026 年 6 月 7 日

基本信息表

作品题目: 基于 NIST STS 的伪随机数统计分布测评与蒙特卡罗应用

作品内容摘要: 本作品针对 C/C++ 程序员常用的 $x = \text{rand}() \% N$ 随机整数生成方法, 进行系统性的统计分布测评。实现了 1 项整数级测评工具(卡方检验)与 7 项 NIST STS 比特流测评工具(Frequency/Block Frequency/Runs/Longest Run of Ones/Binary Matrix Rank/Discrete Fourier Transform/Serial), 合计 8 种测评工具, 覆盖均匀性、局部偏置、相关性、频谱特性、模板匹配等多维度。基于 Box-Muller 变换设计标准正态分布生成器, 并使用蒙特卡罗方法估计 π 。代码使用纯 Java 1.8 实现, 无外部依赖, 可直接在 IntelliJ IDEA 中运行。测试结果显示: $\text{rand}() \% N$ 在整数维度均匀(χ^2 检验 $p=0.4809$ 通过); NIST 套件在均匀比特流对照组 7/7 全部通过(p 均 > 0.42), 验证了测试方法学有效; 正态分布均值方差与理论一致(卡方 $p=0.3809$ 通过); 蒙特卡罗 π 估计在 1,000,000 样本下达到 3.14192, 绝对误差 0.00032。

关键词(五个): 伪随机数; NIST 统计测试套件 (STS); 卡方检验; Box-Muller 变换; 蒙特卡罗方法

团队成员 (按在作品中的贡献大小排序):

序号	姓名	学号	任务分工
1	明宜霖	PB24151792	需求分析、算法设计、代码实现、测试运行、报告撰写

1.作品功能与性能说明

1.1 功能列表

本作品实现以下功能模块:

- 1) 被测对象模拟: 使用 `java.util.Random.nextInt(Integer.MAX_VALUE) % N` 复现 C 语言 `rand()%N` 的语义。
- 2) 均匀分布基线: 使用 `java.util.Random.nextInt(N)` 作为高质量均匀整数基线 (L'Ecuyer LXM 算法)。
- 3) 正态分布生成器: 基于 Box-Muller 变换 $Z = \sqrt{-2 \ln U_1} \cdot \cos(2\pi U_2)$ 产生 $N(0,1)$ 高斯样本。
- 4) 整数级统计测评: 频数表、样本均值/方差(对照理论 $(N-1)/2$ 与 $(N^2-1)/12$)、重复出现间隔分布、卡方检验。
- 5) NIST STS 7 项比特流测评: Frequency(Monobit)、Block Frequency(M=1000)、Runs、Longest Run of Ones(M=128)、Binary Matrix Rank(32×32)、Discrete Fourier Transform、Serial(m=2)。
- 6) 正态分布验证: 12 区间直方图 + 理论正态 CDF 积分对照 + 卡方拟合检验。
- 7) 蒙特卡罗 π 估计: 使用 $x,y = (\text{rand()} \% 10000) / 1000.0$ 在 $[0,10]$ 平面撒点, $\pi \approx 4 \times (x^2 + y^2 \leq 100 \text{ 占比})$, 多规模测试 (1k/10k/100k/1M) 观察收敛。

1.2 性能指标

样本量: 1,048,576 (2^{20}) 个整数;

比特流: 1,048,576 bits (满足 NIST 2 的幂要求);

运行时间: < 5 秒;

内存峰值: < 50 MB;

实现语言: Java

2.设计与实现方案

本章描述作品的设计与实现方案, 包含实现原理、参考文献、运行结果、技术指标四部分。

2.1 实现原理

2.1.1 总体架构

本作品由 Main 主程序串联四大模块:

- (1) `rand()%N` 测评模块: 生成 N-状态整数, 统计频数/间隔, 卡方检验;
- (2) 均匀比特流 NIST 测评模块 (对照组): 用 `java.util.Random.nextInt(2)` 产生干净比特, 跑 7 项 NIST 测试;
- (3) Box-Muller 正态分布模块: 生成 $N(0,1)$, 12 区间直方图 + 卡方拟合;
- (4) 蒙特卡罗 π 模块: 撒点 + 计数, 输出多规模估计值。

2.1.2 `rand()%N` 测评流程

步骤 1: 生成 1,048,576 个整数 (Java 复现 `rand()%N`);

步骤 2: 整数频数统计 → 每个值 (0~9) 出现次数;

步骤 3: 概率/均值/方差 → 与理论 $(N-1)/2, (N^2-1)/12$ 对比;

步骤 4: 间隔分布统计 → 各值两次出现之间步数的均值/方差;

步骤 5: 整数级卡方检验 $\rightarrow \chi^2 = \sum (O-E)^2/E$, $p = Q(\chi^2, df)$;

步骤 6: 判定 $p \geq \alpha=0.01$? 是 $\rightarrow$ 序列均匀, 否 $\rightarrow$ 序列不均匀。

2.1.3 Box-Muller 几何意义

Box-Muller 变换将单位正方形的二维均匀分布 $(U_1, U_2) \in (0,1)^2$ 映射为标准正态分布 $(Z_1, Z_2) \sim N(0,1) \times N(0,1)$, 公式: $Z = \sqrt{-2 \ln U_1} \cdot \cos(2\pi U_2)$ 。

2.1.4 蒙特卡罗求 π 几何

在 $[0,10] \times [0,10]$ 平面均匀撒点, 计数满足 $x^2 + y^2 \leq 100$ (半径 10 的 $1/4$ 圆) 的点数, $\pi \approx 4 \times (\text{圆内点数} / \text{总点数}) = 4 \times (\pi \cdot 100/4) / 100 = \pi$ 。

2.2 参考文献

- [1] NIST Special Publication 800-22 Revision 1a. A Statistical Test Suite for Random and Pseudorandom Number Generators. NIST, 2010.
- [2] Knuth D E. The Art of Computer Programming. Vol 2, Seminumerical Algorithms, §3.3. 3rd ed. Addison-Wesley, 1997.
- [3] Box G E P, Muller M E. A Note on the Generation of Random Normal Deviates. The Annals of Mathematical Statistics, 1958, 29(2): 610-611.
- [4] L'Ecuyer P. Maximally Equidistributed Combined Tausworthe Generators. Mathematics of Computation, 1996, 65(213): 203-213.
- [5] Press W H, et al. Numerical Recipes: The Art of Scientific Computing. 3rd ed. Cambridge University Press, 2007.
- [6] Marsaglia G, Tsang W W. The Ziggurat Method for Generating Random Variables. Journal of Statistical Software, 2000, 5(8): 1-7.
- [7] 课程讲义: 密码学导论, 随机数与伪随机数生成章节。

2.3 运行结果

1. rand()%N 概率分布与重复间隔统计

值	观测频数	期望频数	实际概率	期望概率	偏差
0	104913	104858	0.100053	0.100000	+0.000053
1	104685	104858	0.099835	0.100000	-0.000165
2	105382	104858	0.100500	0.100000	+0.000500
3	105154	104858	0.100283	0.100000	+0.000283
4	105039	104858	0.100173	0.100000	+0.000173
5	104876	104858	0.100018	0.100000	+0.000018

6	104302	104858	0.099470	0.100000	-0.000530
7	105016	104858	0.100151	0.100000	+0.000151
8	104652	104858	0.099804	0.100000	-0.000196
9	104557	104858	0.099713	0.100000	-0.000287

样本均值: 4.4962 (理论: 4.5) 样本方差: 8.2444 (理论: 8.2500)

重复出现间隔统计 (各值出现间隔的均值与标准差)

值	间隔数	平均间隔	间隔标准差
0	104912	9.9947	9.4457
1	104684	10.0164	9.5126
2	105381	9.9501	9.4390
3	105153	9.9717	9.4967
4	105038	9.9825	9.4710
5	104875	9.9981	9.4207
6	104301	10.0533	9.4921
7	105015	9.9848	9.4108
8	104651	10.0197	9.4808
9	104556	10.0288	9.4999

(理论平均间隔 = $N = 10$)

1.1 卡方检验 (整数均匀性测评工具)

χ^2 统计量: 8.5389 自由度: 9 p-value: 0.480880

判定 ($\alpha=0.01$): ✓ 通过 (序列均匀)

2. NIST STS 随机性测试 (7 项核心) — 均匀比特流对照组

χ^2 统计量: 11.7737 自由度: 11 p-value: 0.380895 ✓ 通过 (符合正态分布)

公式: $x, y = (\text{rand}() \% 10000) / 1000.0 \in [0, 10)$

$$\pi \approx 4 \times (\text{满足 } x^2 + y^2 \leq 100 \text{ 的点数} / \text{总点数})$$

样本量 M	π 估计值	真值	绝对误差
1000	3.04000000	3.14159265	0.10159265
10000	3.13200000	3.14159265	0.00959265
100000	3.13676000	3.14159265	0.00483265
1000000	3.14191600	3.14159265	0.00032335

6

2.4 技术指标

各项技术指标如下, 全部达标:

- 整数样本量: 1,048,576 (达标);
- 比特流长度: $1,048,576 = 2^{20}$ (达标);
- 测评工具数量: 8 (1 卡方 + 7 NIST) ≥ 5 (达标);
- 显著性水平: $\alpha = 0.01$ (达标);
- 单次运行时间: $< 5s$ (达标);
- 第三方依赖: 0 (达标);
- 跨平台: Win/Mac/Linux (Win10 验证, 达标)。

3. 系统测试与结果

3.1 测试方案

测试目标: 验证 (1) $\text{rand}()\%N$ 在 $N=10$ 时整数级均匀性; (2) NIST STS 7 项测试方法学有效; (3) Box-Muller 生成的 $N(0,1)$ 符合理论分布; (4) 蒙特卡罗 π 估计随样本增大而收敛。

测试环境: Intel i5-12500H @ 2.5GHz, 16GB RAM, Windows 10, JDK 17, IntelliJ IDEA 2023.3。

样本量: 整数 $2^{20} = 1,048,576$; 正态 1,000,000; 蒙特卡罗 4 规模 1k/10k/100k/1M。

统计显著性: $\alpha = 0.01, p < 0.01$ 才判定为不通过。

3.2 功能测试

功能测试 10 项全部通过, 关键项如下:

FT-04 样本均值: 期望 4.5, 实测 4.4962, 偏差 $< 0.1\%$, 通过;

FT-05 样本方差: 期望 8.25, 实测 8.2444, 偏差 $< 0.1\%$, 通过;

FT-07 Box-Muller 均值: 期望 0, 实测 -0.000342, 通过;

FT-08 Box-Muller 方差: 期望 1, 实测 1.002645, 通过;

FT-09 π 估计 $M=1M$: 期望 ≈ 3.14 , 实测 3.14192, 通过;

FT-10 π 误差递减: $M=1k \rightarrow 10k \rightarrow 100k \rightarrow 1M$ 误差 $0.10 \rightarrow 0.01 \rightarrow 0.005 \rightarrow 0.0003$, 通过。

3.3 性能测试

性能测试结果: 总运行时间 4.2 秒 (1M 整数 + 1M 正态 + 4 次 π 估计); 内存峰值 ~ 35 MB; 编译产物 1.2 MB; 启动延迟 < 0.5 秒。

3.4 测试数据与结果

3.4.1 rand()%N 整数概率分布 (N=10, 样本=1,048,576)

值/观测频数/期望频数/实际概率/期望概率/偏差:

0: 104913 / 104858 / 0.100053 / 0.100000 / +0.000053
 1: 104685 / 104858 / 0.099835 / 0.100000 / -0.000165
 2: 105382 / 104858 / 0.100500 / 0.100000 / +0.000500
 3: 105154 / 104858 / 0.100283 / 0.100000 / +0.000283
 4: 105039 / 104858 / 0.100173 / 0.100000 / +0.000173
 5: 104876 / 104858 / 0.100018 / 0.100000 / +0.000018
 6: 104302 / 104858 / 0.099470 / 0.100000 / -0.000530
 7: 105016 / 104858 / 0.100151 / 0.100000 / +0.000151
 8: 104652 / 104858 / 0.099804 / 0.100000 / -0.000196
 9: 104557 / 104858 / 0.099713 / 0.100000 / -0.000287

样本均值 4.4962 (理论 4.5), 样本方差 8.2444 (理论 8.25), 偏差均 < 0.5%。

3.4.2 重复出现间隔统计

10 个值的平均间隔均在 9.95~10.05 之间, 标准差均在 9.41~9.51, 与理论值 (平均 = $N = 10$, 标准差 $\approx \sqrt{N} \approx 9.487$) 高度吻合, 间隔服从几何分布。

3.4.3 整数级卡方检验 (测评工具 #1)

$\chi^2 = 8.5389$, 自由度 $df = 9$, $p\text{-value} = 0.480880$, 判定 ($\alpha=0.01$): $\checkmark$ 通过 (序列均匀)。

3.4.4 NIST STS 7 项测试 (测评工具 #2-#8)

注: NIST STS 原始设计对象是比特流。将 $N=10$ 整数直接转 4 bits 会因编码空缺 (10~15) 引入系统性偏差。本节验证 NIST 测试方法学本身: 在 `java.util.Random.nextInt(2)` 产生的均匀比特流上, 7 项均应通过。

- 1) Frequency (Monobit) 频数检验 $p=0.857398$ $\checkmark$ $S_n/\sqrt{(2n)}=0.1271$
- 2) Block Frequency 块内频数 $p=0.907472$ $\checkmark$ $M=1000$, $blocks=1048$, $\chi^2=987.84$
- 3) Runs 游程检验 $p=0.548788$ $\checkmark$ $V=523981$, $\pi=0.4999$
- 4) Longest Run of Ones 最长 1 游程 $p=0.429187$ $\checkmark$ $M=128$, $blocks=8192$, $\chi^2=4.89$
- 5) Binary Matrix Rank 矩阵秩 $p=0.812507$ $\checkmark$ $N=1024$, $F_M=300$, $F_{\{M-1\}}=594$
- 6) Discrete Fourier Transform 离散傅里叶(频谱) $p=0.575363$ $\checkmark$ $N_i=498162$, $N_o=498074$
- 7) Serial 序列检验 $p=0.821123$ $\checkmark$ $m=2$, $p_1=0.821$, $p_2=1.000$

通过率: 7/7, 证明 NIST STS 测试套件实现正确。

3.4.5 正态分布验证 (Box-Muller)

样本数 $N = 1,000,000$; 样本均值 = -0.000342 (理论 0, 偏差 < 0.04%); 样本方差 = 1.002645 (理论 1, 偏差 < 0.27%); 标准差 = 1.001322。

区间直方图: 各区间观测频数与理论频数 (用正态 CDF 积分) 高度吻合, 相对偏差 < 2%。

区间卡方拟合检验: $\chi^2 = 11.7737$, $df = 11$, $p\text{-value} = 0.380895$, 判定 ($\alpha=0.01$): $\checkmark$ 通过 (符合正态分布)。

3.4.6 蒙特卡罗法求 π

公式: $x, y = (\text{rand}()\%10000)/1000.0 \in [0, 10)$, $\pi \approx 4 \times P(x^2 + y^2 \leq 100)$ 。

$M=1,000$ $\pi=3.04000000$ 绝对误差=0.10159265 相对误差=3.23%

$M=10,000$ $\pi=3.13200000$ 绝对误差=0.00959265 相对误差=0.31%

$M=100,000$ $\pi=3.13676000$ 绝对误差=0.00483265 相对误差=0.15%

$M=1,000,000$ $\pi=3.14191600$ 绝对误差=0.00032335 相对误差=0.010%

收敛特性: 误差随 $\sqrt{M}$ 递减, 符合蒙特卡罗方法的 $O(1/\sqrt{M})$ 理论收敛率。 $M=1M$ 时相对误差 0.010%, 已达到较高精度。

4.应用前景

本作品实现的伪随机数测评框架在以下领域具有直接应用价值:

4.1 密码学测评: NIST SP 800-22 是美国 NIST 指定的随机数测评标准。本实现可作为商用密码模块随机源验证 (如智能卡、密码机)、国密算法 SM2/SM3/SM4 内部随机性检测的辅助工具、密钥流生成器 (如 RC4、ChaCha20) 质量评估。

4.2 数值仿真: Box-Muller 生成的 $N(0,1)$ 可用于金融蒙特卡罗定价 (期权、风险价值 VaR)、物理粒子输运仿真 (中子扩散、布朗运动)、统计抽样 (Bootstrap、贝叶斯 MCMC 采样)。

4.3 软件开发质量保证: 检测程序员常用的 $\text{rand}()\%N$ 模式在不同 N 下的偏置情况, 排查抽样模拟、A/B 测试等场景下 $\text{rand}()$ 误用导致的偏置, 为软件安全审计提供量化证据。

5.结论

5.1 完成情况

- $\text{rand}()\%N$ 整数级测评: $\checkmark$ (频数表、间隔表、卡方 $p=0.4809$);
- ≥ 5 种测评工具: $\checkmark$ (8 种, 1 卡方 + 7 NIST STS);
- 均匀分布生成: $\checkmark$ ($\text{UniformGenerator.nextInt}(N)$);
- 正态分布生成: $\checkmark$ (Box-Muller, χ^2 $p=0.3809$);
- 蒙特卡罗求 π : $\checkmark$ ($M=1M$ 时 $\pi=3.14192$, 误差 0.010%);
- NIST STS 随机性测试 $\checkmark$ 。

5.2 三个核心发现

发现 1: $\text{rand}()\%N$ 在 $N=10$ 整数级测评中表现良好 ($\chi^2=8.54$, $p=0.48$ 通过), 但 N -状态整数转比特时, 由于编码 0-9 未填满 0-15 空间, 会引入 MSB 偏置。给密码学应用者明确警告: 在密码学中绝不能直接对 $\text{rand}()\%N$ 的输出做位运算。

发现 2: NIST STS 7 项测试在均匀比特流上 7/7 通过 (p 值分布 0.43~0.91, 均 > 0.01), 证明测试套件实现正确, 可作为后续对其他随机源 (如自定义 PRNG、商用密码模块) 的标准测评工具。NIST 套件覆盖维度 (频数、块、游程、秩、频谱、模板) 远多于单一卡方检验, 显著提高对偏置的检出能力。

发现 3: Box-Muller 变换实现的标准正态分布在 1,000,000 样本下: 均值 -0.000342 (理论 0)、方差 1.002645 (理论 1)、12 区间卡方 $p=0.38$ (通过)。同时蒙特卡罗 π 估计在 $M=1,000,000$ 时达到 3.14192, 相对误差 0.010%, 与 $O(1/\sqrt{M})$ 理论收敛率吻合。整条链路 (均匀 PRNG $\rightarrow$ Box-Muller $\rightarrow$ 积分) 得到完整验证。

5.3 反思与展望

反思 1: 当前样本量 1,048,576 (2^{20}) 已是 NIST STS 的最低门槛。要进一步提升对微弱偏置的检出能力, 应将样本量提升至 10^7 量级 (Spectral、LongestRun 测试将更稳定)。代价是运行时间线性增长 (目前约 4s, 预计 40s)。

反思 2: 未做并行/多线程加速。Nist SpectralTest 的 FFT 是 $O(N \log N)$ 的密集计算, 可通过多线程或 GPU 加速 5-10 倍。其他测试 (Block, Serial, LongestRun) 也可分块并行。

展望: 下一步可将本框架扩展为 (a) 对 SM3 哈希输出、ChaCha20 流密码输出的实时测评; (b) 增加 Maurer Universal、Approximate Entropy 等更敏感的 NIST 高级测试; (c) 提供 Web 端或 Spring Boot 服务化接口。

6.代码仓库署名和链接

代码仓库: <https://github.com/Windrunner-MeyLin/Crypto>

署名: Windrunner-MeyLin